

Kapitel 7

Effektivitet
inline eller inte?
Temporärer
Minnespooler

Kapitel 7 - Effektivitet

- **inline**
- När bör man / bör man inte använda **inline**
- Optimering med **inline**
- Temporära objekt
- Minneshantering med minnespool - specialiserad **new** och **delete**

Effektivitet

Innehållet i denna del är inspirerat av

- Scott Meyers, "*Effective C++*"
- Boken nämner 50 specifika tips om hur man bör skriva C++-program samt
- Mayhew och Bulka, "*Efficient C++: performance programming techniques*"
- Boken jämför hastigheten hos olika konstruktioner i C++

Kompileringsprocessen

Kompilering
Länkning

Kompilering till objektfiler

- En **källkodsfil** (med suffix `.cpp`) som är körd genom preprocessor kallas en **översättningsenhet** (*translation unit*)
- I C++ kompileras källkodsfilerna separat till **objektfiler** (med suffix `.o`)

Länkning av objektfiler

- Kompilatorn kan inte veta om anrop sker till funktion som inte är definierad.
- **Länkaren** har översikt över alla definierade funktioner och varnar om anrop sker till funktion som ej är definierad.
- Objektfilerna **länkas** sedan till ett körbart program.

Inline

När är **inline**
olämpligt?
90-10-regeln
Felhantering

Optimering

- Allmänt om optimering
 - Regel nr 1: "Don't do it"
 - Regel nr 2: "Don't do it yet"
- Omöjligt att veta vilka delar som är tidskritiska
- Endast tidtagning avslöjar vilka delar som ska optimeras, om ens då...
Prestanda beror ju även på indata

inline

- **inline** är ett **förslag** till kompilatorn att ersätta funktionsanrop med den kod som anropas
- Man kan tänka sig **inline** som att kompilatorn klipp-och-klistrar kod
- **inline** kan vara till hjälp när man vill skriva effektiva program
- Utan **inline** kan man använda preprocessorn för att uppnå samma effekt. Man får då inte samma typkontroll

inline eller inte?

Lämplighet hos **inline**-definitioner:

- Funktioner med lång exekveringstid (många gånger tiden för anrop) är inte lämpliga för **inline**
- Funktioner som är rekursiva kan vara olämpliga, liksom funktioner med statiska variabler
- Funktionspekare till en funktion som deklarerats **inline** skapar en funktion i minnet som man kan peka till

inline eller inte?

När bör man undvika att använda **inline**?

- Använd **inline** där du lokaliserat en flaskhals
- Överdrivet användande ger märkbart större exekverbara program pga expansion av koden (*code bloat*)
- **inline** kan ge långsammare kod pga sidfel (*page faults*)

inline eller inte?

När bör man undvika att använda **inline**?

- **inline** kan öka kompileringstiderna eftersom programkoden ligger i headerfilerna
- Använda objekt måste finnas definierade
- Ändringar ger omkompilering pga beroenden

inline

Exempel på snabbare program med **inline**:
Optimering mellan funktionsanrop är svårt, med **inline** blir det lättare

```
char foo(int i)
{
    return i % 7;
}

// utan inline och utan optimering
int i = 9;
char c = "abcdefg"[foo(i)];

// med inline och optimering
int i = 9;
char c = 'c';
```

inline och 90-10-regeln

90-10 regeln (även känd som 80-20-regeln):

- De flesta program spenderar 90% av körtiden i 10% av koden
- De funktioner som utför arbetet vill man snabba upp
- Felhanteringskod används sällan och bör inte göras **inline**

inline vid behov

Betrakta följande exempel:

```
void foo() {
    if(error) {
        /* 500 rader felhantering */
    }
    /* 10 rader kod */
}

for(int i = 0; i < 100000; i++)
    foo();    /* foo är tidskritisk

/* foo används på flera andra ställen */
```

inline vid behov

Vi konstaterar följande:

- Funktionen **foo** är tidskritisk
- Gör vi **foo()** **inline** får vi en stor ökning av kodens storlek pga felhanteringsrutinen

Vi väljer att dela upp **foo**:

- Den del av koden som ska exekvera snabbt gör vi **inline**
- Felhanteringen låter vi ske genom funktionsanrop

inline vid behov

Vårt nya, effektiva program blir:

```
inline                                // vi gör foo inline
void foo() {
    if(error) {
        foo_handle_error(); // ej inlineanrop
    }
    /* 10 rader kod */
}
void foo_handle_error() { // felhanteringen ej inline
    /* 500 rader felhantering */
}

for(int i = 0; i < 100000; i++)
    foo(); // foo expanderar till < 15 rader kod
```

Temporära objekt

Onödiga instruktioner
Kopiering kontra
initiering
Referenser

Temporära objekt och kopiering

För att slippa onödiga instruktioner vill man undvika att skapa temporära objekt:

- Vid anrop: skicka **referenser** istället för objekt där så är möjligt.
- Omformulera uttryck för att undvika **temporära objekt**.

Temporära objekt och kopiering

Att slippa temporära objekt:

- Använd **initiering** istället för tilldelning, där så är möjligt
- Använd referenser för att undvika *call-by-value*
- Var medveten om operationer som skapar temporärer

Temporära objekt och kopiering

Exempel på hur man undviker temporära objekt:

```
std::string s;  
std::string r("foo");  
  
s = r + "bar"; // temp. objekt som innehåller "foobar"  
  
s += r;        // bättre: inget temp. objekt  
s += "bar";    // samma resultat
```

Temporära objekt och kopiering

Annat, vanligt förekommande, exempel på onödiga instruktioner

```
class Huge {  
    Huge() : e(0) { heavy(); }  
    Huge(int i) : e(i) { heavy(); }  
    Huge &operator=(const Huge &h)  
    { e = h.e; heavy(); return *this; }  
    heavy() { for(int i = 0; i < 10000000; i++); }  
    int e;  
};  
  
class A {  
    Huge e;  
    A() { e = 7; } // kör heavy tre gånger  
    A(int) : e(7) {} // kör heavy en gång  
};
```

Specialiserad minneshantering

De globala
operatorerna `::new`
och `::delete`
Minnespool för
dynamisk allokering
av små objekt

Specialiserad minneshantering

- De inbyggda **new** och **delete** är byggda för att klara samtliga krav på minnesallokering.
- Eftersom de är generella kan de vara för långsamma i vissa sammanhang.
- Man kan då skapa specialbyggda **new** och **delete** som sköter allokeringen.

Minnespool

- Om man ska allokera många objekt av samma typ kan man minska körtiderna genom att specialisera minnesallokeringen:
- Vi använder den inbyggda **new** för att allokera stora mängder minne; denna mängd kallar vi vår **minnespool**.
- Minnet delas sedan ut i lagom stora portioner i vår egen **new**.

Minnespool - huvudfil

Vi har följande program som vi vill optimera:

```
/* Fraction definierar inte new och delete */
class Fraction {...};

// ineffektivt att allokera 5M objekt ett i taget
for(int i = 0; i < 5000000; i++)
{
    Fraction *f = new Fraction;
    foo(f);
    delete f;
}
```

Minnespool

Vi skapar en mallklass **Pool**:

- Hanterar minnespoolen
- Allokerar större stycken minne för prestanda
- Styckar minne och delar ut till objekt
- Allokerar mer minne vid behov
- Länkar lediga objekt genom att återanvända det minne som objektet ska uppta

Minnespool - headerfil

```
// i filen pool.h
template<class T>
class Pool
{
public:
    Pool();
    ~Pool();
    void *alloc();           // hämta en minnescell
    void free(void *);       // lämna tillbaka minnescell

protected:
    void expand();           // utöka minnet
    enum { EXPAND = 65536 }; // allokerad storlek i bytes
    ...

    std::vector<T *> blocks; // block med minne

    struct List {
        List *next;         // länkad lista av celler
    } freelist;             // pekare till nästa cell
};

#include "pool.inl"         // templateklassens implem.
```

Minnespool - implementation

```
// i filen pool.inl
template<class T>
Pool<T>::Pool() : blocks(100)
{
    assert(sizeof(T) >= sizeof(Pool<T> *));
    freelist.next = 0;
    expand();
}

template<class T>
Pool<T>::~~Pool()
{
    std::vector<T *>::iterator it;
    for(it = blocks.begin(); it != blocks.end(); it++)
        delete [] *it;
}

template<class T>
inline
void *Pool<T>::alloc()
{
    if(!freelist.next)
        expand();

    List *tmp = freelist.next;
    freelist.next = tmp->next;
    return reinterpret_cast<void *>(tmp);
}
```

Minnespool - implementation

```
// i filen pool.inl
template<class T>
inline
void Pool<T>::free(void *p)
{
    List *tmp = reinterpret_cast<List *>(p);
    tmp->next = freelist.next;
    freelist.next = tmp;
}

template<class T>
void Pool<T>::expand()
{
    int size = EXPAND / sizeof(T);
    T *mem = new T[size];
    blocks.push_back(mem);

    List *oldp = reinterpret_cast<List *>(mem);
    oldp->next = freelist.next;
    for(int i = 0; i < size - 1; i++) {
        List *newp =
            reinterpret_cast<List *>(mem + i + 1);
        newp->next = oldp;
        oldp = newp;
    }
    freelist.next = oldp;
}
```

Minnespool - testklass

```
class Fraction
{
public:
    Fraction(int d = 0, int n = 1) : n_(n), d_(d) {}

    void *operator new(size_t size)
    { return pool.alloc(); }
    void operator delete(void *p, size_t)
    { pool.free(p); }

protected:
    int n_;          // nominator
    int d_;          // denominator
    ...

protected:
    static Pool<Fraction> pool; // deklaration
};

Pool<Fraction> Fraction::pool; // definition
```

Minnespool - huvudfil

Den optimerade huvudfilen blir:

```
clock_t start = clock();

for(int i = 0; i < 5000000; i++)
{
    Fraction *f = new Fraction;
    foo(f);
    delete f;
}

std::cout << "testen tog (s): "
<< ((double)clock()-start) / CLOCKS_PER_SEC
<< std::endl;
```


Minnespool - resultat

Mätningar ger:

- Utan minnespool - testen tog (s): 2.37
- Med minnespool - testen tog (s): 0.52
- Hastighetsökning = $2.37/0.52 = 4.56$ ggr